

GeoFS Browser Exploration Guide

Goal

Build a verified, precise map of what the GeoFS website exposes in the browser.

This is not the implementation document for the user's custom controller, aircraft model, wake model, or dynamics model. Those will be written later.

This document is only for reverse-mapping the website structure so future code can be built from verified knowledge.

The target is to document:

1. Global objects exposed by the page
2. Object hierarchy and module structure
3. Readable telemetry fields
4. Aircraft-state fields
5. Control-state fields
6. Discrete command surfaces
7. Continuous-control surfaces
8. Autopilot command surfaces
9. Position/reset/location surfaces
10. Pause/focus/background behavior
11. Frame/update loop hooks
12. Function signatures and side effects
13. Safe vs dangerous writes

This document is verification-driven. Every command should be tested in the browser console before being treated as known.

Ground Rules

Exploration mode

Turn the Tampermonkey bridge script off while exploring.

Reason: if the bridge is active, it may install frame callbacks, open WebSockets, touch autopilot state, or affect control behavior. Exploration should isolate GeoFS's native page objects first.

Browser setup

Use Chrome or Edge DevTools.

Open:

```
https://www.geo-fs.com/geofs.php?v=3.9
```

Open DevTools:

```
F12 or Cmd/Ctrl + Shift + I
```

Use the Console tab first. Later, use Sources for code search and breakpoints.

Rule for documentation

For every discovered API/surface, record:

Field	Meaning
Name	Object or function name
Path	Example: <code>window.geofs.animation.values.kias</code>
Type	number, string, object, function, array
Read/write	read-only, writable, function-call, unknown
Units	ft, kt, deg, rad, normalized input, unknown
Update rate	frame-rate, slower, event-driven, unknown
Verified command	exact console command used
Observed result	what changed or printed
Risk	safe read, mild side effect, high side effect

Phase 1 — Confirm Page Globals

1.1 Confirm GeoFS exists

Run:

```
window.geofs
```

Expected:

A large object. If it prints `undefined`, the sim has not loaded yet or you are on the wrong page/frame.

1.2 Confirm aircraft object exists

Run:

```
window.geofs?.aircraft?.instance
```

Expected:

A large object representing the current aircraft.

1.3 Confirm animation values exist

Run:

```
window.geofs?.animation?.values
```

Expected:

An object containing live simulation / display values such as altitude, heading, speed, vertical speed, etc.

1.4 Confirm controls object exists

Run:

```
window.controls
```

Expected:

A controls-related object. This may include control state, update methods, keyboard/mouse handlers, or command surfaces.

1.5 Confirm autopilot exists

Run:

```
window.geofs?.autopilot
```

Expected:

An autopilot object with state and methods such as mode, values, turn-on/off methods, and target setters.

Phase 2 — Safe Object Inspection

2.1 Print top-level GeoFS keys

Run:

```
Object.keys(window.geofs).sort()
```

Record the result.

2.2 Print top-level aircraft-instance keys

Run:

```
Object.keys(window.geofs.aircraft.instance).sort()
```

Record the result.

2.3 Print animation-value keys

Run:

```
Object.keys(window.geofs.animation.values).sort()
```

Record the result.

2.4 Print controls keys

Run:

```
Object.keys(window.controls).sort()
```

Record the result.

2.5 Print autopilot keys

Run:

```
Object.keys(window.geofs.autopilot).sort()
```

Record the result.

Phase 3 — First Telemetry Probe

Objective

Find stable telemetry fields that can be read at high rate without side effects.

3.1 Snapshot key telemetry fields

Run:

```
((() => {  
  const v = window.geofs?.animation?.values || {};  
  return {  
    altitude: v.altitude,  
    heading360: v.heading360,  
    kias: v.kias,  
    verticalSpeed: v.verticalSpeed,  
    latitude: v.latitude,  
    longitude: v.longitude,  
    groundElevation: v.groundElevation,  
  };  
})())
```

Record which fields exist and which are `undefined`.

3.2 Inspect all numeric animation fields

Run:

```
((() => {  
  const v = window.geofs?.animation?.values || {};  
  return Object.fromEntries(  
    Object.entries(v)  
      .filter(([k, val]) => typeof val === 'number' && Number.isFinite(val))  
      .sort(([a], [b]) => a.localeCompare(b))  
  );  
})())
```

Record the result.

3.3 Determine if values update live

Run while flying:

```
const __probeTelemetry = setInterval(() => {
  const v = window.geofs?.animation?.values || {};
  console.log({
    t: performance.now().toFixed(1),
    altitude: v.altitude,
    heading360: v.heading360,
    kias: v.kias,
    verticalSpeed: v.verticalSpeed,
  });
}, 500);
```

Stop with:

```
clearInterval(__probeTelemetry)
```

Expected:

Values should update over time. If they do, `geofs.animation.values` is a usable telemetry source.

Phase 4 — Frame Callback Probe

Objective

Verify the correct high-rate hook for telemetry.

4.1 Check frame callback function

Run:

```
typeof window.geofs?.api?.addFrameCallback
```

Expected:

```
"function"
```

4.2 Count callback frequency

Run:

```
((() => {
  let count = 0;
  const t0 = performance.now();
```

```

window.geofs.api.addFrameCallback(() => {
  count++;
  const t = performance.now();
  if (t - t0 > 5000 && !window.__frameProbePrinted) {
    window.__frameProbePrinted = true;
    console.log('GeoFS frame callback Hz ≈', count / ((t - t0) / 1000));
  }
});
})();

```

Expected:

A printed approximate callback frequency after about 5 seconds.

Caution:

This installs a callback permanently for the current page session. Refresh the page to remove it.

Phase 5 — Discovered Surfaces

Verified from browser-console inspection.

Surface	Path	Type	Purpose	Verified?	Notes
Main GeoFS object	<code>window.geofs</code>	object	Root sim object	Yes	Contains ~180 top-level keys. Important roots: <code>aircraft</code> , <code>animation</code> , <code>api</code> , <code>autopilot</code> , <code>camera</code> , <code>doPause</code> , <code>flyTo</code> , <code>resetFlight</code> , <code>togglePause</code> , <code>undoPause</code> .
Aircraft instance	<code>window.geofs.aircraft.instance</code>	object	Current aircraft physical/ model state	Yes	Contains ~52 keys. Important fields: <code>llaLocation</code> , <code>lastLlaLocation</code> , <code>htr</code> , <code>velocity</code> , <code>airVelocity</code> , <code>rigidBody</code> , <code>controllers</code> , <code>definition</code> , <code>engine</code> , <code>engines</code> , <code>groundContact</code> , <code>crashed</code> .

Surface	Path	Type	Purpose	Verified?	Notes
Animation values	<code>window.geofs.animation.values</code>	object	Primary readable telemetry	Yes	Contains ~166 keys. This is currently the cleanest fast telemetry source.
Controls object	<code>window.controls</code>	object	Input/control state and command surface	Yes	Contains ~81 keys. Important roots: <code>setters</code> , <code>axisSetters</code> , <code>states</code> , <code>pitch</code> , <code>roll</code> , <code>yaw</code> , <code>throttle</code> , <code>flaps</code> , <code>gear</code> , <code>airbrakes</code> , <code>brakes</code> , <code>update</code> .
Autopilot object	<code>window.geofs.autopilot</code>	object	Built-in autopilot state and setters	Yes	Contains ~22 keys. Methods include <code>turnOn</code> , <code>turnOff</code> , <code>toggle</code> , <code>setCourse</code> , <code>setAltitude</code> , <code>setSpeed</code> , <code>setVerticalSpeed</code> , <code>setMode</code> .
Frame callback	<code>window.geofs.api.addFrameCallback</code>	function	High-rate hook	Not yet	Needs explicit frequency probe.

Phase 5A — Current Interpretation

Primary telemetry source

Use:

```
window.geofs.animation.values
```

This appears to be the best high-level telemetry object. It already includes aircraft attitude, speed, altitude, gear/flap state, throttle, forces/lifts, load factor, Mach, ground speed, vertical speed, and environmental fields.

Important observed fields:

Field	Meaning	Observed unit / interpretation
<code>altitude</code>	altitude	feet

Field	Meaning	Observed unit / interpretation
<code>altitudeMeters</code>	altitude	meters
<code>haglFeet</code>	height above ground level	feet
<code>haglMeters</code>	height above ground level	meters
<code>heading</code>	signed heading	degrees, may be negative
<code>heading360</code>	wrapped heading	degrees, 0–360
<code>kias</code>	indicated airspeed	knots
<code>ktas</code>	true airspeed	knots, appears same in current snapshot
<code>groundSpeedKnt</code>	ground speed	knots
<code>groundSpeed</code>	ground speed	likely m/s
<code>airspeedms</code>	airspeed	m/s
<code>verticalSpeed</code>	vertical speed	likely m/s, not ft/min
<code>climbrate</code>	climb rate	appears same as <code>verticalSpeed</code>
<code>pitch</code>	displayed pitch	degrees
<code>roll</code>	displayed roll	degrees
<code>yaw</code>	yaw	degrees or internal angle; verify later
<code>aoa</code>	angle of attack	degrees
<code>mach</code>	Mach number	dimensionless
<code>loadFactor</code>	load factor	g
<code>throttle</code>	throttle	0–1
<code>brakes</code>	brakes	likely 0–1
<code>flapsPosition</code>	actual flap position	step/position
<code>flapsTarget</code>	target flap setting	step/position
<code>flapsSteps</code>	number of flap steps	count
<code>gearPosition</code>	actual gear position	0–1
<code>gearTarget</code>	target gear state	0–1
<code>parkingBrake</code>	parking brake	boolean
<code>enginesOn</code>	engines on	boolean
<code>groundContact</code>	ground contact	boolean/0–1
<code>stalling</code>	stall state	boolean
<code>windSpeed</code>	wind	likely knots or internal unit; verify later

Lower-level physical state source

Use:

```
window.geofs.aircraft.instance
```

This is closer to the actual simulation state. Important observed fields:

Field	Meaning
<code>llaLocation</code>	aircraft latitude, longitude, altitude in meters
<code>lastLlaLocation</code>	previous aircraft location
<code>htr</code>	heading/tilt/roll-style attitude array
<code>velocity</code>	velocity vector
<code>airVelocity</code>	air-relative velocity vector
<code>rigidBody</code>	lower-level physics object
<code>controllers</code>	aircraft controller objects for pitch/roll/yaw
<code>engine</code> , <code>engines</code>	engine state
<code>groundSpeed</code> , <code>trueAirSpeed</code> , <code>velocityScalar</code>	scalar speed fields
<code>definition</code> , <code>setup</code>	aircraft definition/config

Control write candidates

Do not write yet. First inspect.

Likely surfaces:

```
window.controls.setters
window.controls.axisSetters
window.controls.states
window.controls.flaps
window.controls.gear
window.controls.airbrakes
window.controls.brakes
window.controls.pitch
window.controls.roll
window.controls.yaw
window.controls.throttle
```

Autopilot command candidates

These are already callable methods, but should be tested carefully:

```
window.geofs.autopilot.turnOn()  
window.geofs.autopilot.turnOff()  
window.geofs.autopilot.setCourse(deg)  
window.geofs.autopilot.setAltitude(ft)  
window.geofs.autopilot.setSpeed(kts)  
window.geofs.autopilot.setVerticalSpeed(value)  
window.geofs.autopilot.setMode(mode)
```

|---|---:|---| | Main GeoFS object | `window.geofs` | object | Root sim object | No | | | Aircraft
instance | `window.geofs.aircraft.instance` | object | Current aircraft state/model | No | | |
Animation values | `window.geofs.animation.values` | object | Display/live telemetry | No | | |
Frame callback | `window.geofs.api.addFrameCallback` | function | High-rate hook | No | | |
Controls object | `window.controls` | object | User control handling | No | | | Autopilot object |
`window.geofs.autopilot` | object | Built-in autopilot | No | |

Phase 6 — Website Structure Mapping

Objective

Map the website's own structure thoroughly before writing custom code.

We are not designing the custom model here. We are discovering the native browser-side API surface that future code may use.

Current known root objects

Verified root surfaces:

```
window.geofs  
window.geofs.aircraft.instance  
window.geofs.animation.values  
window.controls  
window.geofs.autopilot
```

Likely important children:

```
window.geofs.api  
window.geofs.camera  
window.geofs.map
```

```
window.geofs.flightPlan
window.geofs.runways
window.geofs.nav
window.geofs.preferences
window.geofs.frameCallbackStack
window.geofs.aircraftList
window.controls.setters
window.controls.axisSetters
window.controls.states
```

Exploration method

For each object:

1. List keys
2. Classify each key as data / function / DOM / unknown
3. Inspect function source using `String(fn)`
4. Probe arity using `fn.length`
5. Search for related names in DevTools Sources
6. Test read-only behavior first
7. Test low-risk functions only after signature is understood
8. Record result in table

Standard object-inspection helper

Use this helper in the console:

```
window.__describeObject = function describeObject(obj) {
  return Object.fromEntries(
    Object.entries(obj || {}).
      sort(([a], [b]) => a.localeCompare(b)).
      map(([k, v]) => [
        k,
        {
          type: typeof v,
          isArray: Array.isArray(v),
          keys: v && typeof v === 'object' && !Array.isArray(v)
            ? Object.keys(v).slice(0, 20).sort()
            : null,
          length: typeof v === 'function' ? v.length : null,
          source: typeof v === 'function' ? String(v).slice(0, 300) : null,
        }
      ])
  );
};
```

Then inspect objects with:

```
__describeObject(window.geofs)
__describeObject(window.geofs.api)
__describeObject(window.geofs.aircraft.instance)
__describeObject(window.geofs.animation.values)
__describeObject(window.controls)
__describeObject(window.controls.setters)
__describeObject(window.controls.axisSetters)
__describeObject(window.geofs.autopilot)
```

Controls mapping — verified stage 1

`window.controls.setters`

Verified as an object containing 35 discrete command descriptors.

Observed keys:

```
[
  'cameraDown',
  'cameraLeft',
  'cameraRight',
  'cameraUp',
  'cycleFlaps',
  'decreaseThrottle',
  'fullReverse',
  'increaseThrottle',
  'lookLeft',
  'lookRight',
  'none',
  'pushToTalk',
  'reset',
  'setAccessories',
  'setAirbrakes',
  'setAirbrakesDown',
  'setAirbrakesUp',
  'setBrakes',
  'setElevatorTrimDown',
  'setElevatorTrimNeutral',
  'setElevatorTrimUp',
  'setFlaps',
  'setFlapsDown',
  'setFlapsUp',
  'setGear',
  'setGearDown',
  'setGearUp',
```

```

    'setOptionalAnimatedPart',
    'setReverseFull',
    'setReverseNone',
    'setReverseOnThrottleAxis',
    'toggleAutoPilot',
    'toggleEngines',
    'toggleParkingBrake',
    'toggleReverse'
]

```

Interpretation:

`controls.setters` appears to describe button/discrete actions, not direct bare functions. Each entry is an object. The next step is to inspect the object fields inside individual setters.

Likely categories:

Category	Keys
Camera/view	cameraDown, cameraLeft, cameraRight, cameraUp, lookLeft, lookRight
Throttle/reverse	increaseThrottle, decreaseThrottle, fullReverse, toggleReverse, setReverseFull, setReverseNone, setReverseOnThrottleAxis
Aircraft configuration	setGear, setGearUp, setGearDown, setFlaps, setFlapsUp, setFlapsDown, cycleFlaps, setAirbrakes, setAirbrakesUp, setAirbrakesDown, setBrakes, toggleParkingBrake
Trim	setElevatorTrimUp, setElevatorTrimDown, setElevatorTrimNeutral
Systems	toggleEngines, toggleAutoPilot, pushToTalk, reset
Misc model animation	setAccessories, setOptionalAnimatedPart

`window.controls.axisSetters`

Verified as an object containing 14 continuous/axis command descriptors.

Observed keys:

```

[
  'airbrakesPosition',
  'brakes',
  'flapsPosition',
  'hatView',
  'increaseThrottle',
  'lookAround',

```

```

    'lookUpDown',
    'none',
    'pitch',
    'reverse',
    'roll',
    'throttle',
    'throttlerreverse',
    'yaw'
]

```

Interpretation:

`controls.axisSetters` appears to describe continuous axis inputs. Each entry is also an object, not a bare function.

Likely categories:

Category	Keys
Primary flight controls	<code>pitch</code> , <code>roll</code> , <code>yaw</code>
Propulsion	<code>throttle</code> , <code>reverse</code> , <code>throttlerreverse</code> , <code>increaseThrottle</code>
Drag/braking	<code>brakes</code> , <code>airbrakesPosition</code>
Configuration	<code>flapsPosition</code>
View axes	<code>hatView</code> , <code>lookAround</code> , <code>lookUpDown</code>

Controls mapping — verified stage 3: exact command semantics

Discrete command descriptors

`window.controls.setters` entries are button-style command descriptors.

General shape:

```

{
  label: string,
  set: function(),
  unset?: function(),
  overridesAutopilot?: boolean,
  overridesPeer?: boolean
}

```

Verified examples:

Command	Call	Exact behavior
<code>setGear</code>	<code>window.controls.setters.setGear.set()</code>	Toggles <code>controls.gear.target</code> between <code>0</code> and <code>1</code> . If <code>controls.gear.target</code> is <code>1</code> , then calls <code>controls.setPartAnimationDelta(1)</code> . Only allowed if gear operation is allowed, aircraft is not on ground, or debug is on.
<code>setFlaps</code>	<code>window.controls.setters.setFlaps.set()</code>	If <code>controls.flaps.target</code> is valid, calls <code>controls.setPartAnimationDelta(1)</code> through <code>definition.flapsPosition</code> .
<code>setBrakes</code>	<code>.set()</code> / <code>.unset()</code>	<code>.set()</code> sets <code>geofs.aircraft.instance.brakesOn</code> to <code>1</code> and <code>controls.brakes = 1</code> . <code>.unset()</code> sets <code>geofs.aircraft.instance.brakesOn</code> to <code>0</code> and <code>controls.brakes = 0</code> .
<code>toggleParkingBrake</code>	<code>.set()</code>	Toggles <code>geofs.aircraft.instance.brakesOn</code> to <code>0</code> or <code>1</code> .
<code>toggleEngines</code>	<code>.set()</code>	Calls <code>geofs.aircraft.instance.stopEngine()</code> if engine is on, else <code>startEngine()</code> .
<code>reset</code>	<code>.set()</code>	Calls <code>geofs.resetFlight()</code> .

Exact source snippets observed:

```
// reset
function(){geofs.resetFlight()}

// setBrakes
function(){geofs.aircraft.instance.brakesOn=!0,controls.brakes=1}

// unsetBrakes
function(){geofs.aircraft.instance.brakesOn=!1,controls.brakes=0}

// toggleParkingBrake
function(){
  geofs.aircraft.instance.brakesOn
    ? (geofs.aircraft.instance.brakesOn=!1, controls.brakes=0)
    : (geofs.aircraft.instance.brakesOn=!0, controls.brakes=1)
}

// toggleEngines
function(){
  geofs.aircraft.instance.engine.on
    ? geofs.aircraft.instance.stopEngine()
    : geofs.aircraft.instance.startEngine()
}
```

Continuous axis descriptors

`window.controls.axisSetters` entries are continuous command descriptors.

General shape:

```
{
  label: string,
  process: function(value),
  overridesAutopilot?: boolean,
  overridesPeer?: boolean
}
```

Verified examples:

Axis	Call	Input range interpretation	Exact behavior
<code>pitch</code>	<code>window.controls.axisSetters.pitch.process(e)</code>	likely <code>[-1, +1]</code>	<code>controls.geofs.pitch</code>
<code>roll</code>	<code>window.controls.axisSetters.roll.process(e)</code>	likely <code>[-1, +1]</code>	<code>controls.geofs.roll</code>
<code>yaw</code>	<code>window.controls.axisSetters.yaw.process(e)</code>	likely <code>[-1, +1]</code>	<code>controls.geofs.yaw</code>
<code>throttle</code>	<code>window.controls.axisSetters.throttle.process(e)</code>	<code>[-1, +1]</code>	<code>controls.throttle</code>
<code>brakes</code>	<code>window.controls.axisSetters.brakes.process(e)</code>	likely <code>[0, 1]</code> or axis raw	<code>controls.brakes</code>
<code>flapsPosition</code>	<code>window.controls.axisSetters.flapsPosition.process(e)</code>	<code>[-1, +1]</code>	<code>controls.flapsPosition</code>

Exact source snippets observed:

```
// pitch
/e/ => controls.rawPitch = e * geofs.preferences.joystick.sensitivity

// roll
/e/ => controls.roll = e * geofs.preferences.joystick.sensitivity

// yaw
/e/ => controls.yaw = e * geofs.preferences.joystick.sensitivity

// throttle
/e/ => controls.throttle = (e + 1) / 2
```

```
// brakes
/e/ => controls.brakes = e

// flapsPosition
process(e){
  controls.flaps.positionTarget = (e + 1) / 2 * controls.flaps.maxPosition;
  controls.setPartAnimationDelta(controls.flaps);
}
```

Correction: the arrow functions are displayed by the browser as `e=>...`; they are not regex syntax. The table above is the reliable interpretation.

Immediate implications

For continuous-control writes

The simplest direct-control path appears to be:

```
window.controls.axisSetters.pitch.process(e)
window.controls.axisSetters.roll.process(e)
window.controls.axisSetters.yaw.process(e)
window.controls.axisSetters.throttle.process(e)
```

Where:

```
pitch/roll/yaw: e is multiplied by joystick sensitivity
throttle: e = -1 maps to 0 throttle; e = +1 maps to 1 throttle
```

For button/discrete writes

The simplest discrete-control path appears to be:

```
window.controls.setters.setGear.set()
window.controls.setters.setBrakes.set()
window.controls.setters.setBrakes.unset()
window.controls.setters.toggleParkingBrake.set()
window.controls.setters.toggleEngines.set()
```

Avoid for now

Do not call yet unless intentionally testing side effects:

```
window.controls.setters.reset.set()
window.geofs.resetFlight()
```

Controls mapping — verified stage 4: update-loop behavior

```
window.controls.update(e)
```

This is the main control update function.

Observed structure:

```
controls.update(e)
├─ controls.updateKeyboard(e)
├─ controls.copilot.update(e)
├─ if controls.mode == 'joystick': controls.updateJoystick(e)
├─ if autopilot is OFF:
│   └─ apply trim key states
│       └─ if mouse/touch mode and no keyboard override: controls.updateMouse(e)
│       └─ if orientation mode: controls.updateOrientation(e)
│       └─ if touch mode: controls.updateTouch(e)
│       └─ apply exponential shaping
│       └─ apply pitch/roll/yaw multipliers
│       └─ compute rawYaw
│       └─ optional mixYawRoll
│       └─ compute steering
│       └─ apply yaw exponential
├─ clamp roll/rawPitch/yaw to [-1, +1]
├─ controls.pitch = controls.rawPitch + controls.elevatorTrim
└─ continue throttle/reverse/brakes/configuration logic
```

Important exact snippets

Observed `updateMouse`:

```
function(e){
  controls.roll = controls.mouse.xValue *
geofs.preferences.mouse.sensitivity;
  controls.rawPitch = controls.mouse.yValue *
geofs.preferences.mouse.sensitivity;
}
```

Observed `recenter`:

```
function(){
  controls.mouse.xValue = 0;
  controls.mouse.yValue = 0;
  controls.yaw = 0;
  controls.roll = 0;
  controls.rawPitch = 0;
}
```

Observed final update behavior includes:

```
controls.roll = clamp(controls.roll, -1, 1);
controls.rawPitch = clamp(controls.rawPitch, -1, 1);
controls.yaw = clamp(controls.yaw, -1, 1);
controls.pitch = controls.rawPitch + controls.elevatorTrim;
```

Keyboard update behavior

`controls.updateKeyboard(e)` increments/decrements:

```
controls.roll
controls.rawPitch
controls.yaw
```

based on key states:

```
controls.states.left
controls.states.right
controls.states.up
controls.states.down
controls.states.rudderLeft
controls.states.rudderRight
```

It also recenters axes if the aircraft controller definition asks for recentering.

Joystick update behavior

`controls.updateJoystick(e)` loops over joystick axes, maps each physical joystick axis through:

```
geofs.preferences.joystick.axis[t]
```

Then obtains a descriptor from:

```
controls.axisSetters[o]
```

It calls:

```
r.process(s, e)
```

only if conditions allow it:

```
(!geofs.autopilot.on || r.overridesAutopilot)  
and peer-sharing/recorder conditions allow it
```

Critical implication for future injection

Direct writes to:

```
controls.rawPitch  
controls.roll  
controls.yaw  
controls.throttle
```

can be overwritten by:

```
controls.updateMouse(e)  
controls.updateKeyboard(e)  
controls.updateJoystick(e)
```

especially before the clamp/final `controls.pitch = controls.rawPitch + controls.elevatorTrim` step.

For future Tampermonkey continuous-control injection, likely strategies are:

Strategy	Meaning	Risk
Inject after <code>controls.update()</code>	Set final values after native input update	Medium
Replace mouse values	Write <code>controls.mouse.xValue/yValue</code> before <code>updateMouse</code>	Medium
Use joystick axis descriptor path	Call <code>controls.axisSetters.*.process(e)</code> at the right time	Medium
Disable/avoid native input mode	Prevent mouse/keyboard from overwriting injected values	Medium-high
Patch <code>controls.update</code> wrapper	Wrap native update and inject after it	High, but clean for later

Current safe conclusion

For documentation only:

The native control update loop owns the final values each frame.
Manual injection must either feed into that loop or run after that loop.

GeoFS API mapping — verified stage 1

`window.geofs.api`

`window.geofs.api` is a large browser-side rendering/world utility API. It appears to be mostly Cesium/world/rendering/model/camera/terrain utilities, not the primary aircraft-physics API.

Important categories discovered:

Category	Representative keys	Usefulness
Frame loop	<code>addFrameCallback</code> , <code>removeFrameCallback</code> , <code>frameCallbackWrapper</code> , <code>precisionTime</code>	Very high
Model/world insertion	<code>loadModel</code> , <code>Model</code> , <code>addModelToWorld</code> , <code>removeModelFromWorld</code> , <code>destroyModel</code> , <code>addModelInstance</code> , <code>commitInstanceCollection</code>	Very high
Model transforms	<code>setModelPositionOrientationAndScale</code> , <code>setModelRotationPosition</code> , <code>setModelVisibility</code> , <code>setModelOpacity</code> , <code>setModelColor</code> , <code>toggleModelShadow</code>	Very high
Model node transforms	<code>getModelNode</code> , <code>getNodePosition</code> , <code>getNodeRotation</code> , <code>setNodeRotationTranslationScale</code> , <code>setNodeScale</code> , <code>setNodeVisibility</code>	High
Labels/ billboards/ overlays	<code>addLabel</code> , <code>removeLabel</code> , <code>setLabelPosition</code> , <code>updateLabelText</code> , <code>billboard</code> , <code>groundTexture</code>	High
Camera	<code>camera</code> , <code>getCameraLla</code> , <code>setCameraLookAt</code> , <code>setCameraPositionAndOrientation</code> , <code>getFOV</code> , <code>setFOV</code> , <code>getHeading</code> , <code>getTilt</code>	High
Coordinate/ screen picking	<code>getScreenCoordFromLla</code> , <code>getLlaFromScreencoordDepth</code> , <code>getPositionFromScreenCoords</code> , <code>getDistanceFromScreenCoords</code> , <code>getModelFromScreenCoords</code> , <code>getNodeNameFromScreenCoords</code> , <code>xyz2lla</code>	High
Terrain/ ground	<code>getFastTerrainElevation</code> , <code>getGroundAltitude</code> , <code>getGuaranteedGroundAltitude</code> , <code>getGroundNormal</code>	High

Category	Representative keys	Usefulness
Rendering/ imagery	<code>renderingQuality</code> , <code>setImageryProvider</code> , <code>setGoogleTiles</code> , <code>setTerrainProvider</code> , <code>setBuildings</code> , <code>setVegetation</code> , <code>setWaterEffect</code> , <code>triggerExplicitRendering</code>	Medium
Imagery/ color	<code>setImageryBrightness</code> , <code>setImageryContrast</code> , <code>setImageryGamma</code> , <code>setImageryHue</code> , <code>setImagerySaturation</code> , <code>setImageryColorModifier</code> , <code>setAtmosphereColorModifier</code>	Low- medium
Platform/ input capability	<code>hasTouch</code> , <code>hasOrientation</code> , <code>isWebXRAvailable</code> , <code>testIfMobileDevice</code>	Low- medium

Critical API functions

`addFrameCallback(callback, namespace?, maxExecutionTime?)`

Observed function length:

3

Observed source:

```
function(e,t,a){
  return t=t||"global",
  geofs.frameCallbackStack[t] ||
    (geofs.frameCallbackStack[t]={
      callbacks:{},
      lastId:0,
      maxExecutionTime:a,
      lastIndex:1
    }),
  geofs.frameCallbackStack[t].lastId++,

  geofs.frameCallbackStack[t].callbacks[geofs.frameCallbackStack[t].lastId]=e,
  geofs.frameCallbackStack[t].lastId
}
```

Interpretation:

```
const id = geofs.api.addFrameCallback(fn, namespace, maxExecutionTime);
```

Default namespace:

```
"global"
```

It stores callbacks in:

```
geofs.frameCallbackStack[namespace].callbacks[id]
```

```
removeFrameCallback(id, namespace?)
```

Observed function length:

```
2
```

Observed source:

```
function(e,t){  
  t=t||"global";  
  geofs.frameCallbackStack[t] && delete  
  geofs.frameCallbackStack[t].callbacks[e]  
}
```

Interpretation:

```
geofs.api.removeFrameCallback(id, namespace);
```

Default namespace:

```
"global"
```

Model/world API

```
loadModel(optionsOrUrl)
```

Observed as async function with length `1`.

Purpose appears to be loading a Cesium model from URL/options.

```
new geofs.api.Model(urlOrName, options)
```

Observed constructor length `2`.

Appears to wrap `geofs.api.loadModel(options)` and create a model wrapper with promises.

`addModelToWorld(model)`

Observed function length 1.

Behavior:

```
If model.instance: add model instance
Else if model.entity: viewer.entities.add(model.entity)
Else: geofs.api.models.add(model)
```

`removeModelFromWorld(model)`

Observed function length 1.

Behavior:

```
If model.entity: viewer.entities.remove(model.entity)
Else: geofs.api.models.remove(model)
```

`destroyModel(model)`

Observed function length 1.

Behavior:

```
removeModelFromWorld(model)
then call model.destroy() if available and not already destroyed
```

Model transform API

`setModelPositionOrientationAndScale(model, lla, htr?, scale?)`

Observed function length:

4

Important because this appears to be the main API for placing a model at a latitude/longitude/altitude with heading/tilt/roll and scale.

Likely argument meaning:

```
geofs.api.setModelPositionOrientationAndScale(model, lla, htr, scale)
```

Where:

```
lla = [lat_deg, lon_deg, alt_m]
htr = [heading_deg, tilt_or_pitch_deg, roll_deg]
scale = scalar or [sx, sy, sz]
```

Needs later confirmation with a harmless label/model test.

Other useful model functions

```
geofs.api.setModelVisibility(model, visible)
geofs.api.setModelOpacity(model, alpha)
geofs.api.setModelColor(model, r, g, b, a)
geofs.api.setModelElevation(model, altitude)
geofs.api.setModelRotationPosition(model, rotationMatrix, lla)
```

Label API

```
addLabel(text, lla?, options?)
```

Observed function length:

```
3
```

Observed behavior:

```
position = Cesium.Cartesian3.fromDegrees(lon, lat, alt)
text = geofs.api.makeLabelTextSafe(text)
return geofs.api.labels.add(options)
```

Call form:

```
const label = geofs.api.addLabel("test", [lat, lon, alt], options);
```

```
setLabelPosition(label, lla)
```

Observed length `2`.

```
updateLabelText(label, text)
```

Observed length `2`.

```
removeLabel(label)
```

Observed length `1`.

This is a good low-risk way to test coordinate placement before testing models.

Camera and coordinate utilities

Useful functions:

```
geofs.api.getCameraLla(camera)
geofs.api.setCameraLookAt(camera, lla)
geofs.api.setCameraPositionAndOrientation(camera, lla, htr)
geofs.api.getScreenCoordFromLla(lla)
geofs.api.getLlaFromScreencoordDepth(x, y, depth)
geofs.api.getPositionFromScreenCoords(x, y)
geofs.api.getModelFromScreenCoords(x, y)
geofs.api.getNodeNameFromScreenCoords(x, y)
geofs.api.xyz2lla(xyz, referenceLla)
```

Important convention confirmed in source:

```
Cesium.Cartesian3.fromDegrees(lla[1], lla[0], lla[2])
```

So GeoFS `lla` order is:

```
[latitude_deg, longitude_deg, altitude_m]
```

Cesium internally wants:

```
longitude, latitude, altitude
```

Terrain / ground utilities

Useful functions:

```
geofs.api.getFastTerrainElevation(lla)
geofs.api.getGroundAltitude(lla, options?)
geofs.api.getGuaranteedGroundAltitude(lla)
geofs.api.getGroundNormal(lla, options?)
```

These are useful for documenting ground clearance, spawn/reset altitude checks, and future location placement.

Next API mapping probes

Frame callback stack — verified

Observed structure:

```
geofs.frameCallbackStack = {
  global: {
    callbacks: {},
    lastId: 1,
    lastIndex: 1
  },
  docProbe: {
    callbacks: {},
    lastId: 1,
    lastIndex: 1
  }
}
```

Observed `frameCallbackWrapper` behavior:

```
function(e,t){
  for (var a in geofs.api.precisionTime = geofs.utils.now(),
geofs.frameCallbackStack) {
    var o = geofs.frameCallbackStack[a];
    if (o.maxExecutionTime > 0) {
      var n = o.lastIndex;
      do {
        try {
          o.callbacks[n](geofs.api.precisionTime)
        } catch(r) {
          geofs.debug.error(r, "geofs.api.frameCallbackWrapper
maxExecutionTime");
          geofs.debug.throw(r)
        }
        ++n > o.lastId && (n = 1)
      } while (geofs.api.precisionTime + o.maxExecutionTime >
geofs.utils.now());
      o.lastIndex = n
    } else {
      for (var n in o.callbacks) {
        try {
          o.callbacks[n](geofs.api.precisionTime)
        } catch(s) {
          geofs.debug.error(s, "geofs.api.frameCallbackWrapper general");

```

```

        geofs.debug.throw(s)
    }
}
}
}
}
}

```

Interpretation:

Every render/frame callback cycle:

1. `geofs.api.precisionTime` is updated using `geofs.utils.now()`
2. Each namespace in `geofs.frameCallbackStack` is visited
3. If `maxExecutionTime > 0`, callbacks are executed round-robin until time budget expires
4. Otherwise, all callbacks in that namespace are executed
5. Callback receives `geofs.api.precisionTime` as first argument

Verified add/remove API:

```

const id = geofs.api.addFrameCallback(callback, namespace, maxExecutionTime);
geofs.api.removeFrameCallback(id, namespace);

```

Default namespace if omitted:

```
"global"
```

Frame callback frequency — verified

Test code registered a callback in namespace:

```
"docProbe"
```

Result:

```

{
  namespace: "docProbe",
  frames: 127,
  seconds: 2.0010999999046324,
  hz: 63.465094201215585
}

```

Verified current callback rate:

~63.47 Hz

Important notes:

This is browser/session dependent.
It may change with render load, tab focus, browser throttling, monitor refresh, and GeoFS rendering settings.
But the callback mechanism itself is verified.

Recommended future telemetry hook

For telemetry documentation/testing, use:

```
const ns = "telemetryProbe";
const id = geofs.api.addFrameCallback((t) => {
  const aircraft = geofs.aircraft.instance;
  const values = geofs.animation.values;
  // read only here
}, ns);

// later
geofs.api.removeFrameCallback(id, ns);
```

Label placement test — verified

A low-risk label was inserted above the current aircraft position.

Test code used:

```
((() => {
  const lla = window.geofs.aircraft.instance.llaLocation.slice();
  lla[2] += 100;

  const label = geofs.api.addLabel("DOC_PROBE", lla, {
    font: "14px sans-serif"
  });

  window.__docProbeLabel = label;

  return {
    label,
    lla,
    screen: geofs.api.getScreenCoordFromLla(lla)
  };
})()
```

Observed result:

```
{
  lla: [
    37.303987664379,
    69.134729694696,
    6195.996577907743
  ],
  screen: {
    x: 431.82687744131937,
    y: -683.6279025002036
  }
}
```

Verified:

```
geofs.api.addLabel(text, lla, options) works
geofs.api.getScreenCoordFromLla(lla) works
GeoFS accepts lla order as [lat_deg, lon_deg, alt_m]
```

The returned label object is a Cesium label-like object with fields including:

```
_text
_font
_fillColor
_outlineColor
_position
_scale
_heightReference
_glyphs
```

The negative screen `y` value means the computed label point was above/outside the current viewport, not that the world coordinate failed.

Remove label with:

```
geofs.api.removeLabel(window.__docProbeLabel)
```

Aircraft state and telemetry — verified stage 1

Main telemetry roots

Two useful telemetry roots are confirmed:

```
geofs.aircraft.instance
geofs.animation.values
```

Interpretation:

Root	Meaning	Best use
<code>geofs.aircraft.instance</code>	Main live aircraft object; contains state, physics objects, engines, controllers, parts, models, rigid body, definition	Raw/live state and deeper reverse engineering
<code>geofs.animation.values</code>	Large derived/instrument/animation telemetry dictionary	Fast read-only telemetry export

`geofs.aircraft.instance` important keys

Observed keys include:

```
llaLocation
lastLlaLocation
htr
velocity
velocityScalar
velocityDirection
airVelocity
airVelocityDirection
trueAirSpeed
groundSpeed
relativeAltitude
angleOfAttackDeg
htrAngularSpeed
rigidBody
controllers
definition
engine
engines
parts
models
object3d
groundContact
waterContact
brakesOn
crashed
stalling
totalThrust
maxAngularVRatio
```

Current observed values:

```
geofs.aircraft.instance.llaLocation = [  
    37.14540988060616,  
    68.80703264240177,  
    409.086842132968  
]  
  
geofs.aircraft.instance.htr = [  
    -126.61,  
    -6.3,  
    -4.09  
]  
  
geofs.aircraft.instance.velocity = [  
    -88.85846887529404,  
    -65.308395251472,  
    3.4200270945553948  
]
```

Likely interpretation:

```
llaLocation = [lat_deg, lon_deg, alt_m]  
htr = [heading_deg, tilt/pitch_deg, roll_deg]  
velocity = 3D velocity vector, likely local/world GeoFS coordinates; exact  
frame still needs confirmation
```

Aircraft definition keys

`geofs.aircraft.instance.definition` contains aircraft configuration/dynamics data, including:

```
mass  
originalInertia  
tensorFactor  
dragFactor  
minimumSpeed  
motionSensitivity  
autopilot  
cameras  
parts  
contactProperties  
flapsSteps  
flapsTravelTime  
gearTravelTime  
airbrakesTravelTime  
engineInertia  
maxRPM  
minRPM
```

```
zeroThrustAltitude
Vspeeds
```

This is likely the aircraft-model definition surface, not the live telemetry surface.

`geofs.animation.values` telemetry keys

`geofs.animation.values` is a large telemetry dictionary. It contains approximately 160+ keys.

High-value telemetry fields:

Position / altitude / ground

```
altitude
altitudeMeters
haglMeters
haglFeet
groundElevationFeet
groundContact
```

Observed sample:

```
altitude: 1401.432808191122      // likely feet
altitudeMeters: 427.1567192873757
haglMeters: 70.82615826460665
haglFeet: 232.36928599823625
groundElevationFeet: 1168.6972504998055
groundContact: 0
```

Attitude / heading

```
heading
heading360
pitch
roll
yaw
atilt
aroll
rawPitch
rawYaw
turnrate
```

Observed sample:

```
heading: -125.71628109355812
heading360: 234.2837189064419
pitch: 0.17343429908497568
roll: 0.012550826688679434
yaw: -0.015077108620643345
atilt: -7.3399314521017125
aroll: -3.0458859137022696
turnrate: 12.105495559578989
```

Important distinction:

```
geofs.aircraft.instance.htr appears to be physical aircraft orientation in
degrees.
geofs.animation.values.pitch/roll/yaw appear to be control/input/animation
values, not necessarily Euler attitude degrees.
geofs.animation.values.atilt/aroll appear closer to aircraft visual attitude
angles.
```

Needs confirmation.

Speed / air data

```
kias
ktas
airspeedms
groundSpeed
groundSpeedKnt
mach
verticalSpeed
climbrate
kiasChangeRate
```

Observed sample:

```
kias: 222.96814016449173
ktas: 222.96814016449173
airspeedms: 114.70472113975111
groundSpeed: 114.49837247533293
groundSpeedKnt: 222.5670304501436
mach: 0.3325843910592441
verticalSpeed: 1356.5618322119537
climbrate: 1356.5618322119537
```

Likely units:

```
kias, ktas, groundSpeedKnt: knots  
airspeedms, groundSpeed: m/s  
verticalSpeed / climbrate: likely ft/min
```

Acceleration / jerk / angular acceleration

```
acceleration  
accX  
accY  
accZ  
jerk  
jerkX  
jerkY  
jerkZ  
smoothJerkZ  
angularAcceleration  
aAccX  
aAccY  
aAccZ  
loadFactor  
slipball
```

Observed sample:

```
acceleration: [-0.10098983619649882, 2.5309242252894606, 10.454693170754371]  
jerk: [-0.040124329283123386, -0.022999953504944395, -0.14446216410181406]  
angularAcceleration: [0.0019980976876795586, 0.0004039146756748478,  
0.00019916918697074075]  
loadFactor: 1.0657179582828105  
slipball: -0.10062824335456838
```

Aero / forces / aircraft state

```
aoa  
stalling  
bodyLift  
leftWingLift  
rightWingLift  
rudderLift  
stabilizerLift  
verticalStabLift  
aileronleftLift  
aileronRightLift  
flapLeftLift  
flapRightLift
```

```
thrust
totalThrust on aircraft.instance
```

Observed sample:

```
aoa: 6.840550569125345
stalling: false
bodyLift: -19087.85297357974
leftWingLift: 45418.976445022396
rightWingLift: 45687.35807107185
thrust: 1005464.3676221851
```

Note: lift/thrust units are not yet confirmed.

Controls / configuration

```
throttle
smoothThrottle
brakes
parkingBrake
gearPosition
gearTarget
flapsPosition
flapsTarget
flapsSteps
flapsPositionRatio
airbrakesPosition
airbrakesTarget
trim
steering
```

Observed sample:

```
throttle: 1
smoothThrottle: 0.9999999999999973
brakes: 0
parkingBrake: false
gearPosition: 0.72537222222248717
gearTarget: 1
flapsPosition: 0
flapsTarget: 0
flapsSteps: 6
trim: 0
```

Environment / time

```
windSpeed  
windSpeedLabel  
airTemp  
season  
night  
hours  
minutes  
compHours  
compMinutes  
compSeconds  
geofsTime
```

Observed sample:

```
windSpeed: 0  
windSpeedLabel: "0 kts"  
airTemp: 22.42420698210929  
season: "spring"  
night: false  
hours: 12  
minutes: 0  
geofsTime: 6347645.399999976
```

Recommended telemetry read path

For first documentation/export version, read:

```
const a = geofs.aircraft.instance;  
const v = geofs.animation.values;  
  
const telemetry = {  
  time: geofs.api.precisionTime,  
  lla: a.llaLocation,  
  htr: a.htr,  
  velocity: a.velocity,  
  airVelocity: a.airVelocity,  
  trueAirSpeed: a.trueAirSpeed,  
  groundSpeed: a.groundSpeed,  
  altitudeMeters: v.altitudeMeters,  
  altitudeFeet: v.altitude,  
  haglMeters: v.haglMeters,  
  heading360: v.heading360,  
  kias: v.kias,  
  ktas: v.ktas,
```

```

    airspeedMS: v.airspeedms,
    groundSpeedKnt: v.groundSpeedKnt,
    verticalSpeed: v.verticalSpeed,
    aoa: v.aoa,
    loadFactor: v.loadFactor,
    acceleration: v.acceleration,
    angularAcceleration: v.angularAcceleration,
    throttle: v.throttle,
    brakes: v.brakes,
    gearPosition: v.gearPosition,
    flapsPosition: v.flapsPosition,
    stalling: v.stalling,
    crashed: a.crashed,
    groundContact: a.groundContact
  };

```

Aircraft rigid-body / physics state — verified stage 2

Main rigid-body root

Confirmed:

```
const rb = geofs.aircraft.instance.rigidBody;
```

Observed rigid-body keys:

```

gravityForce
impulseQueue
m_worldInvInertiaTensor
mass
minAngularVelocity
minLinearVelocity
s_inverseMass
v_acceleration
v_angularAcceleration
v_angularVelocity
v_jerk
v_linearVelocity
v_localInvInertia
v_prevAcceleration
v_prevAngularVelocity
v_prevLinearVelocity
v_totalForce
v_totalTorque

```

This is the best source for lower-level physics state.

Confirmed rigid-body fields

Field	Meaning	Observed type
<code>rb.mass</code>	aircraft rigid-body mass	number
<code>rb.s_inverseMass</code>	inverse mass	number
<code>rb.v_linearVelocity</code>	linear velocity vector	array length 3
<code>rb.v_prevLinearVelocity</code>	previous linear velocity vector	array length 3
<code>rb.v_acceleration</code>	acceleration vector	array length 3
<code>rb.v_prevAcceleration</code>	previous acceleration vector	array length 3
<code>rb.v_jerk</code>	jerk vector	array length 3
<code>rb.v_angularVelocity</code>	angular velocity vector	array length 3
<code>rb.v_prevAngularVelocity</code>	previous angular velocity vector	array length 3
<code>rb.v_angularAcceleration</code>	angular acceleration vector	array length 3
<code>rb.v_totalForce</code>	total force vector	array length 3
<code>rb.v_totalTorque</code>	total torque vector	array length 3
<code>rb.v_localInvInertia</code>	local inverse inertia vector/tensor representation	array length 3
<code>rb.m_worldInvInertiaTensor</code>	world inverse inertia tensor representation	array length 3
<code>rb.gravityForce</code>	gravity force vector	array length 3
<code>rb.impulseQueue</code>	pending impulses	array
<code>rb.minAngularVelocity</code>	angular velocity threshold	number
<code>rb.minLinearVelocity</code>	linear velocity threshold	number

Observed constants:

```
rb.mass = 300000
rb.s_inverseMass = 0.0000033333333333333333
rb.minAngularVelocity = 0.01
rb.minLinearVelocity = 0.1
```

Mass/inverse-mass consistency:

```
1 / 300000 = 0.0000033333333333333333
```

So `rb.mass` and `rb.s_inverseMass` are internally consistent.

Selected aircraft state — verified sample

Observed:

```
geofs.aircraft.instance.llaLocation = [
  37.13933641381086,
  68.79624327911588,
  484.8241895654442
]

geofs.aircraft.instance.lastLlaLocation = [
  37.13935073477444,
  68.7962694449603,
  484.6054732342729
]

geofs.aircraft.instance.htr = [
  -124.79923472082338,
  -7.0342635827218745,
  -1.9053292575766758
]

geofs.aircraft.instance.velocity = [
  -101.5206894074644,
  -69.40555689208266,
  9.551131570825781
]

geofs.aircraft.instance.velocityScalar = 123.34831097161485

geofs.aircraft.instance.velocityDirection = [
  -0.8230407745982556,
  -0.5626794266202347,
```

```
0.07743220394013912  
]
```

Velocity consistency check:

```
velocityScalar ≈ norm(velocity)  
velocityDirection ≈ velocity / velocityScalar
```

So the velocity vector is coherent.

Observed airspeed/ground speed:

```
trueAirSpeed: 123.34831097161485  
groundSpeed: 122.97797243924205
```

Because wind was previously observed as zero, `airVelocity` matched `velocity` in this sample.

Angular-rate availability

Confirmed angular velocity exists here:

```
geofs.aircraft.instance.rigidBody.v_angularVelocity
```

Confirmed angular acceleration exists here:

```
geofs.aircraft.instance.rigidBody.v_angularAcceleration
```

`geofs.aircraft.instance.htrAngularSpeed` was observed as:

```
[0, 0, 0]
```

So for angular-rate telemetry, prefer:

```
geofs.aircraft.instance.rigidBody.v_angularVelocity
```

not:

```
geofs.aircraft.instance.htrAngularSpeed
```

until proven otherwise.

Controllers object

Confirmed:

```
geofs.aircraft.instance.controllers.pitch  
geofs.aircraft.instance.controllers.roll  
geofs.aircraft.instance.controllers.yaw
```

Each has:

```
ratio  
recenter  
sensitivity
```

This matches earlier `controls.updateKeyboard(e)` behavior, where keyboard pitch uses aircraft-specific controller ratio/sensitivity and optional recentering.

Engine object

Observed engine fields:

```
geofs.aircraft.instance.engine = {  
  on: true,  
  startup: false,  
  rpm: 9999,  
  invRPMRange: 0.00011111111111111112  
}
```

Engine on/off path already verified:

```
geofs.aircraft.instance.engine.on  
  ? geofs.aircraft.instance.stopEngine()  
  : geofs.aircraft.instance.startEngine()
```

Updated recommended telemetry packet

For serious read-only telemetry, use this packet:

```
const a = geofs.aircraft.instance;  
const rb = a.rigidBody;
```

```

const v = geofs.animation.values;

const telemetry = {
  time: geofs.api.precisionTime,

  // navigation/state
  lla: a.llaLocation?.slice(),
  lastLla: a.lastLlaLocation?.slice(),
  htr: a.htr?.slice(),
  velocity: a.velocity?.slice(),
  velocityScalar: a.velocityScalar,
  velocityDirection: a.velocityDirection?.slice(),
  airVelocity: a.airVelocity?.slice(),
  trueAirSpeed: a.trueAirSpeed,
  groundSpeed: a.groundSpeed,

  // rigid body
  rbLinearVelocity: rb.v_linearVelocity?.slice(),
  rbAngularVelocity: rb.v_angularVelocity?.slice(),
  rbAcceleration: rb.v_acceleration?.slice(),
  rbAngularAcceleration: rb.v_angularAcceleration?.slice(),
  rbJerk: rb.v_jerk?.slice(),
  rbTotalForce: rb.v_totalForce?.slice(),
  rbTotalTorque: rb.v_totalTorque?.slice(),
  rbMass: rb.mass,

  // instruments/derived
  altitudeMeters: v.altitudeMeters,
  altitudeFeet: v.altitude,
  haglMeters: v.haglMeters,
  heading360: v.heading360,
  kias: v.kias,
  ktas: v.ktas,
  airspeedMS: v.airspeedms,
  groundSpeedKnt: v.groundSpeedKnt,
  verticalSpeed: v.verticalSpeed,
  aoa: v.aoa,
  loadFactor: v.loadFactor,

  // controls/state
  throttle: v.throttle,
  brakes: v.brakes,
  gearPosition: v.gearPosition,
  flapsPosition: v.flapsPosition,
  stalling: v.stalling,
  crashed: a.crashed,
  groundContact: a.groundContact
};

```

Rigid-body values and frame convention — verified stage 3

Sample snapshot

Observed aircraft state:

```
aircraft.lla = [  
  37.132823852624526,  
  68.78419817325884,  
  570.4983939111654  
]  
  
aircraft.htr = [  
  -124.29656442373452,  
  -4.93625688291416,  
  -1.1401461672666753  
]  
  
aircraft.velocity = [  
  -113.79739477855578,  
  -76.26500693446532,  
  7.733742800716694  
]  
  
aircraft.velocityScalar = 137.20790472421098
```

Observed rigid-body state:

```
rb.v_linearVelocity = [  
  -113.81480381523406,  
  -76.27564907150027,  
  7.732635931495806  
]  
  
rb.v_acceleration = [  
  -1.2132573640114739,  
  -0.7416105306457546,  
  9.732474161000392  
]  
  
rb.v_angularVelocity = [  
  -0.0019166515694185365,  
  0.0013390401352125821,  
  0.0005125823835003237  
]  
  
rb.v_angularAcceleration = [  
  -0.00044229366966092655,  
  -0.0005731357660332795,
```

```
-0.000011088655350055977  
]
```

Observed animation state:

```
animation.acceleration = [  
  -0.11933833047270406,  
  2.252517675463597,  
  9.572595672038402  
]  
  
animation.angularAcceleration = [  
  -0.00026897840973813113,  
  0.00006858287148066152,  
  -0.00005151274820839619  
]  
  
animation.heading360 = 235.70343557626546  
animation.atilt = -4.93625688291416  
animation.aroll = -1.1401461672666753  
animation.turnrate = 1.5159324167888675  
animation.kias = 266.71082958260246  
animation.groundSpeed = 136.98977458591762  
animation.verticalSpeed = 1522.1809323391876  
animation.aoa = 4.714202065662033
```

Verified relationships

1. `aircraft.velocity` and `rb.v_linearVelocity` are nearly identical

```
aircraft.velocity ≈ rb.v_linearVelocity
```

Difference in sample:

```
rb.linearVelocity - aircraft.velocity ≈ [  
  -0.01740903667828,  
  -0.01064213703495,  
  -0.00110686922089  
]
```

Conclusion:

For velocity telemetry, either `aircraft.velocity` or `rb.v_linearVelocity` works.

Use `aircraft.velocity` for clean high-level telemetry.
Use `rb.v_linearVelocity` when also reading `rb` angular velocity/force/torque.

2. `aircraft.velocityScalar` is the norm of `aircraft.velocity`

$$\text{velocityScalar} \approx ||\text{velocity}||$$

3. `aircraft.velocityDirection` is normalized velocity

$$\text{velocityDirection} \approx \text{velocity} / \text{velocityScalar}$$

4. `animation.atilt/aroll` match `aircraft.htr[1]/htr[2]`

Observed:

```
aircraft.htr[1] = -4.93625688291416
animation.atilt = -4.93625688291416

aircraft.htr[2] = -1.1401461672666753
animation.aroll = -1.1401461672666753
```

Conclusion:

```
animation.atilt = aircraft physical tilt/pitch angle in degrees
animation.aroll = aircraft physical roll angle in degrees
```

5. `heading360` is wrapped heading from `htr[0]`

Observed:

```
aircraft.htr[0] = -124.29656442373452
animation.heading360 = 235.70343557626546
```

Relationship:

$$-124.29656442373452 + 360 = 235.70343557626548$$

Conclusion:

```
animation.heading360 = wrap360(aircraft.htr[0])
```

Important acceleration distinction

`rb.v_acceleration` and `animation.acceleration` are not equal.

Observed:

```
rb.v_acceleration = [-1.2133, -0.7416, 9.7325]
animation.acceleration = [-0.1193, 2.2525, 9.5726]
```

Conclusion:

```
Use rb.v_acceleration for rigid-body physics state.
Use animation.acceleration for instrument/animation-derived acceleration.
They are not the same frame or not the same processed signal.
```

Same for angular acceleration:

```
rb.v_angularAcceleration != animation.angularAcceleration
```

For raw dynamics:

```
rb.v_acceleration
rb.v_angularVelocity
rb.v_angularAcceleration
```

For instrument-like telemetry:

```
geofs.animation.values.acceleration
geofs.animation.values.angularAcceleration
```

Force/torque observation

Observed:

```
rb.v_totalForce = [0, 0, 0]
rb.v_totalTorque = [0, 0, 0]
rb.gravityForce = [0, 0, -2943000]
```

Because:

```
mass = 300000
gravityForce.z = -2943000
2943000 / 300000 = 9.81
```

Conclusion:

```
gravityForce is consistent with SI-like m/s2 gravity and mass units.
rb.v_totalForce and rb.v_totalTorque may be cleared after integration each
frame, so a normal console snapshot can show zero even while acceleration is
nonzero.
```

Inertia observation

Observed:

```
rb.v_localInvInertia = [
  1.6666666666666668e-7,
  1.6666666666666668e-7,
  1.6666666666666668e-7
]

rb.m_worldInvInertiaTensor = [
  [1.6666666666666668e-7, 0, 0],
  [0, 1.6666666666666668e-7, 0],
  [0, 0, 1.6666666666666668e-7]
]
```

This implies isotropic inverse inertia in the current aircraft/sample.

Updated best telemetry recommendations

Physical attitude

Use:

```
headingDeg = geofs.animation.values.heading360;
tiltDeg = geofs.animation.values.atilt;
rollDeg = geofs.animation.values.aroll;
```

or raw:

```
htr = geofs.aircraft.instance.htr;
```

Physical velocity

Use:

```
velocity = geofs.aircraft.instance.velocity;  
speedMS = geofs.aircraft.instance.velocityScalar;
```

or rigid-body:

```
velocity = geofs.aircraft.instance.rigidBody.v_linearVelocity;
```

Angular rates

Use:

```
angularVelocity = geofs.aircraft.instance.rigidBody.v_angularVelocity;
```

Raw dynamics acceleration

Use:

```
acceleration = geofs.aircraft.instance.rigidBody.v_acceleration;  
angularAcceleration =  
geofs.aircraft.instance.rigidBody.v_angularAcceleration;
```

Instrument acceleration

Use:

```
instrumentAcceleration = geofs.animation.values.acceleration;  
instrumentAngularAcceleration = geofs.animation.values.angularAcceleration;
```

Next targeted probe: determine velocity frame axes

Need a short time-series while flying roughly straight. This will tell whether velocity vector components are local ENU-like, NED-like, or another GeoFS convention.

Run this for about 3 seconds:

```

(() => {
  const ns = "axisProbe";
  const samples = [];
  let id = null;
  let t0 = performance.now();

  id = geofs.api.addFrameCallback((t) => {
    const a = geofs.aircraft.instance;
    const v = geofs.animation.values;
    samples.push({
      dt: (performance.now() - t0) / 1000,
      lla: a.llaLocation?.slice(),
      htr: a.htr?.slice(),
      velocity: a.velocity?.slice(),
      rbVelocity: a.rigidBody?.v_linearVelocity?.slice(),
      heading360: v.heading360,
      groundSpeed: v.groundSpeed,
      verticalSpeed: v.verticalSpeed,
      atilt: v.atilt,
      aroll: v.aroll
    });

    if (performance.now() - t0 > 3000) {
      geofs.api.removeFrameCallback(id, ns);
      console.log(JSON.stringify(samples, null, 2));
    }
  }, ns);

  return { id, namespace: ns };
})();

```

Need from output:

First 5 samples
Last 5 samples

Purpose:

Compare finite-difference lla motion with velocity vector.
Determine approximate axis convention and altitude sign.

3. GeoFS API

Purpose: discover official-ish functions exposed by GeoFS.

Commands:

```
Object.keys(window.geofs.api).sort()  
__describeObject(window.geofs.api)
```

4. Frame callback stack

Purpose: understand update-loop registration and frequency.

Commands:

```
window.geofs.frameCallbackStack  
__describeObject(window.geofs.frameCallbackStack)  
String(window.geofs.frameCallback).slice(0, 1000)
```

5. Pause / background behavior

Purpose: find native pause/focus controls.

Commands:

```
{  
  pause: window.geofs.pause,  
  pauseLevel: window.geofs.pauseLevel,  
  pauses: window.geofs.pauses,  
  userPause: window.geofs.userPause,  
  isPaused_type: typeof window.geofs.isPaused,  
  doPause_len: window.geofs.doPause?.length,  
  undoPause_len: window.geofs.undoPause?.length,  
  togglePause_len: window.geofs.togglePause?.length,  
  isPaused_source: String(window.geofs.isPaused).slice(0, 300),  
  doPause_source: String(window.geofs.doPause).slice(0, 500),  
  undoPause_source: String(window.geofs.undoPause).slice(0, 500),  
  togglePause_source: String(window.geofs.togglePause).slice(0, 500),  
}
```

6. Position / reset / fly-to functions

Purpose: discover location reset and teleport-style functions without calling yet.

Commands:

```
{  
  resetFlight_type: typeof window.geofs.resetFlight,  
  resetFlight_len: window.geofs.resetFlight?.length,  
  resetFlight_source: String(window.geofs.resetFlight).slice(0, 1000),  
  flyTo_type: typeof window.geofs.flyTo,  
  flyTo_len: window.geofs.flyTo?.length,  
}
```

```
flyTo_source: String(window.geofs.flyTo).slice(0, 1000),  
initialCoordinates: window.geofs.initialCoordinates,  
lastFlightCoordinates: window.geofs.lastFlightCoordinates,  
}
```

Phase 7 — To Be Explored Later

Discrete writes

Examples to investigate later:

```
flaps  
gear  
brakes  
spoilers  
pause  
reset  
view/camera
```

Continuous controls

Examples to investigate later:

```
pitch  
yaw  
roll  
throttle  
rudder  
elevator  
aileron
```

Aircraft reset / teleport

Need to identify whether GeoFS exposes position setters through:

```
aircraft instance  
simulation model  
location service  
mission/startup helpers
```

Do not write position fields blindly until the state model is understood.

Background behavior

Need to test whether throttling comes from:

```
browser requestAnimationFrame throttling
GeoFS pause-on-blur logic
Page Visibility API
input focus handling
network/sim update loop
```

Current Working Architecture

```
GeoFS page globals
  ↓ inspected safely
Verified telemetry/controls map
  ↓ documented precisely
Tampermonkey bridge
  ↓ JSON messages
Local controller/autopilot process
```